

# Smart Contract Security Review

## PasswordStore Audit

## Findings

### [H-1] Sensitive Data Stored On-Chain (Password Disclosure via Storage Inspection)

#### Description:

The `PasswordStore::s_password` state variable is marked as private, which prevents direct access from other contracts. However, this does not provide true secrecy, as all contract storage is publicly readable on-chain.

An attacker can directly inspect the contract storage and retrieve the stored password without needing any access to the contract's functions.

#### Impact:

Any external user can read the stored password directly from blockchain storage without being the owner. This breaks the assumption that private state variables provide confidentiality and results in complete exposure of sensitive data.

#### Proof of Concept:

Deploy the contract:

```
make deploy
```

This returns a deployed contract address:

```
0x5FbDB2315678afecb367f032d93F642f64180aa3
```

Read storage slot 1 (where `s_password` is stored):

```
cast storage 0x5FbDB2315678afecb367f032d93F642f64180aa3 --rpc-url http://127.0.0.1:8545 1
```

Output:

[illegible]

Convert the bytes32 value to string:

[illegible]

Result:

myPassword

This shows that sensitive data is publicly readable despite being marked private.

## [H-2] Missing Access Control on Password Update Function

**Description:**

The setPassword function does not implement any access control mechanism. As a result, any external address can modify the stored password.

```
function setPassword(string memory newPassword) external {
    s_password = newPassword;
    emit SetNetPassword();
}
```

**Impact:**

Any user can overwrite the stored password, leading to unauthorized modification of sensitive state. This breaks the intended ownership model and allows malicious actors to disrupt or manipulate contract behavior.

### Proof of Concept:

```

function test_IfNonCanSetPassword() public {
    // Arrange
    address randomUser = makeAddr("randomUser");
    string memory expectedPassword = "newPassword";

    // Act
    vm.prank(randomUser);
    passwordStore.setPassword(expectedPassword);

    vm.prank(owner);
    string memory actualPassword = passwordStore.getPassword();

    // Assert
    assertEq(actualPassword, expectedPassword);
}---

```

```

title: "PasswordStore Smart Contract Audit Report"
auditor: "Rabby"
date: "2026-05-06"
logo: "password-store-logo.png"
---
```

# Smart Contract Security Review

## PasswordStore Audit

---

# Findings

### [H-1] Sensitive Data Stored On-Chain (Password Disclosure via Storage Inspection)

**\*\*Description:\*\***

The `PasswordStore::s\_password` state variable is marked as private, which prevents direct access.

An attacker can directly inspect the contract storage and retrieve the stored password without requiring any special permissions.

**\*\*Impact:\*\***

Any external user can read the stored password directly from blockchain storage without being able to interact with the contract.

**\*\*Proof of Concept:\*\***

Deploy the contract:

```
`make deploy`
```

This returns a deployed contract address:

```
0x5FbDB2315678afecb367f032d93F642f64180aa3`
```

Read **storage** slot **1** (where `s_password` **is** stored):

```
`cast storage 0x5FbDB2315678afecb367f032d93F642f64180aa3 --rpc-url http://127.0.0.1:8545 1`
```

Output:

```
0x6d7950617373776f72640000000000000000000000000000000000000000000014`
```

Convert the `bytes32` value to `string`:

```
`cast parse-bytes32-string 0x6d7950617373776f72644000000000000000000000000000000000000000000014`
```

Result:

```
`myPassword`
```

This shows that sensitive data **is** publicly readable despite being marked **private**.

### ### [H-2] Missing Access Control on Password Update Function

**\*\*Description:\*\***

The `setPassword` function does not implement any access control mechanism. As a result, any external

```

    solidity
function setPassword(string memory newPassword) external {
    s_password = newPassword;
    emit SetNetPassword();
}

```

**Impact:**

Any user can overwrite the stored password, leading to unauthorized modification of sensitive state. This breaks the intended ownership model and allows malicious actors to disrupt or manipulate contract behavior.

## Proof of Concept:

```
function test_IfNonCanSetPassword() public {
    // Arrange
    address randomUser = makeAddr("randomUser");
    string memory expectedPassword = "newPassword";

    // Act
    vm.prank(randomUser);
    passwordStore.setPassword(expectedPassword);

    vm.prank(owner);
    string memory actualPassword = passwordStore.getPassword();

    // Assert
    assertEq(actualPassword, expectedPassword);
}
```

Test result:

[PASS] test\_IfNonCanSetPassword() (gas: 30861)

Logs:

Password set by random user: newPassword

This confirms that any user can successfully overwrite the password.

## Recommended Mitigation:

Add an access control modifier to setPassword

Example fix:

```
function setPassword(string memory newPassword) external {
    + => if (msg.sender != s_owner)
    + => {revert error PasswordStore__NotOwner();}
    s_password = newPassword;
    emit SetNetPassword();
}
```

Test result:

[PASS] test\_IfNonCanSetPassword() (gas: 30861)

Logs:

Password set by random user: newPassword

This confirms that any user can successfully overwrite the password.

**\*\*Recommended Mitigation:\*\***

Add an access control modifier to setPassword

Example fix:

```
``` diff
function setPassword(string memory newPassword) external {
+ => if (msg.sender != s_owner)
+ => {revert  error PasswordStore__NotOwner();}
    s_password = newPassword;
    emit SetNetPassword();
}
```